



دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس مبانی رایانش توزیع شده
تمرین سوم (۳)

استاد درس

محمدرضا شکورنیا

دستیار ارشد

پویا جمشیدی

دستیاران مسئول تمرین

مبینا حقی زاده

محمد مهدی ابراهیم سلطانی

بهار ۱۴۰۵

فهرست مطالب

۳	۱ موضوع تمرین
۳	۲ هدف تمرین
۳	۳ شرح کلی تمرین
۴	۴ بخش اول: سوالات تئوری
۴	۵ بخش دوم: پیاده سازی Store Key-Value Replicated
۶	۶ بخش سوم: نسخه گذاری و حل تعارض
۷	۷ بخش چهارم: آزمایش و تحلیل
۸	۸ متریک های مورد نیاز
۹	۹ فناوری های مجاز
۹	۱۰ ساختار پیشنهادی تحویل
۱۰	۱۱ گزارش نهایی
۱۱	۱۲ نمونه جدول گزارش
۱۱	۱۳ معیار ارزیابی
۱۲	۱۴ کسری نمره
۱۳	۱۵ دستاوردهای تمرین

۱ موضوع تمرین

موضوع این تمرین آشنایی عملی با مفاهیم *Replication* و *Consistency* در سیستم های توزیع شده است. در این تمرین باید یک سیستم ساده *Replicated Key-Value Store* طراحی و پیاده سازی کنید و رفتار آن را تحت مدل های مختلف سازگاری بررسی نمایید.

۲ هدف تمرین

اهداف اصلی این تمرین عبارت اند از:

- درک دلیل استفاده از *replication* در سیستم های توزیع شده
- مشاهده عملی مشکل ناسازگاری بین *replica* ها
- پیاده سازی یک مکانیزم ساده تکثیر داده
- مقایسه *strong consistency* و *eventual consistency*
- بررسی اثر تاخیر شبکه، خرابی *replica* و خواندن از *replica* های مختلف
- تحلیل *trade-off* بین سازگاری، دسترس پذیری و کارایی

۳ شرح کلی تمرین

شما باید یک *Key-Value Store* ساده پیاده سازی کنید که شامل چند *replica* مستقل باشد. هر *replica* باید به صورت یک فرآیند یا سرویس جداگانه اجرا شود و داده های خود را نگهداری کند. کلاینت باید بتواند عملیات زیر را انجام دهد:

• *PUT key value*

• *GET key*

سیستم باید حداقل سه replica داشته باشد:

Replica 1

Replica 2

Replica 3

۴ بخش اول: سوالات تئوری

در این بخش باید یک گزارش کوتاه ولی دقیق درباره مفاهیم زیر بنویسید:

- چرا داده ها در سیستم های توزیع شده replicate می شوند؟
- تفاوت replication برای افزایش کارایی و replication برای تحمل خطا چیست؟
- مشکل اصلی replication از نظر consistency چیست؟
- تفاوت strong consistency و eventual consistency چیست؟
- مفهوم read-your-writes consistency چیست؟
- مفهوم monotonic reads چیست؟
- چرا در سیستم های توزیع شده نمی توان همزمان همیشه بیشترین consistency و availability و tolerance partition را به صورت کامل داشت؟

۵ بخش دوم: پیاده سازی Store Key-Value Replicated

۱-۵ نیازمندی های اصلی

در این بخش باید یک سیستم ساده شامل حداقل سه replica پیاده سازی کنید.

۱. هر replica باید یک سرویس مستقل باشد.

۲. هر replica باید داده های خود را به صورت محلی نگهداری کند.
۳. کلاینت باید بتواند به یکی از replica ها درخواست بفرستد.
۴. هر replica باید عملیات *GET* و *PUT* را پشتیبانی کند.
۵. پس از عملیات *PUT*، مقدار جدید باید به replica های دیگر ارسال شود.
۶. ارتباط بین replica ها باید از طریق شبکه انجام شود.
۷. اجرای همه عملیات فقط در یک متغیر مشترک داخل یک برنامه قابل قبول نیست.

۲-۵ مدل اول: Consistency Eventual

در مدل اول، عملیات *PUT* ابتدا روی replica دریافت کننده اعمال می شود و سپس به صورت غیرهمزمان به replica های دیگر ارسال می گردد. رفتار مورد انتظار:

- پس از *PUT*، ممکن است برخی replica ها مقدار جدید را هنوز نداشته باشند.
- اگر بلافاصله از replica دیگر *GET* انجام شود، ممکن است مقدار قدیمی برگردد.
- پس از گذشت زمان، همه replica ها باید به مقدار یکسان برسند.

۳-۵ مدل دوم: Consistency Strong ساده شده

در مدل دوم، عملیات *PUT* فقط زمانی موفق محسوب می شود که حداقل اکثریت replica ها مقدار جدید را تایید کرده باشند. برای سه replica اکثریت برابر با دو replica است. رفتار مورد انتظار:

- *PUT* باید به حداقل دو replica اعمال شود.

- اگر اکثریت در دسترس نباشد، عملیات *PUT* باید شکست بخورد.
- *GET* باید از replica اصلی یا از مسیر هماهنگ شده انجام شود تا مقدار ناسازگار برنگردد.

۶ بخش سوم: نسخه گذاری و حل تعارض

برای هر مقدار ذخیره شده باید یک نسخه نگهداری شود.
حداقل ساختار داده پیشنهادی:

```
{  
  "key": "x",  
  "value": "10",  
  "version": 3,  
  "updated_by": "replica1"  
}
```

۱-۶ نیازمندی ها

- هر بار که مقدار یک کلید تغییر می کند، نسخه آن افزایش یابد.
- اگر یک replica مقدار قدیمی تر دریافت کرد، نباید مقدار جدیدتر خود را با مقدار قدیمی جایگزین کند.
- اگر دو مقدار با نسخه یکسان ولی مقدار متفاوت دریافت شد، باید یک سیاست حل تعارض مشخص داشته باشید.

۲-۶ سیاست حل تعارض پیشنهادی

برای ساده سازی، می توانید از یکی از روش های زیر استفاده کنید:

- *Last-Write-Wins*

- انتخاب مقدار replica با شناسه بزرگ تر
- نگهداری هر دو مقدار و گزارش conflict

۷ بخش چهارم: آزمایش و تحلیل

باید چند سناریوی آزمایشی طراحی و اجرا کنید.

۱-۷ سناریوی ۱: مشاهده ناسازگاری موقت

۱. روی *Replica 1* مقدار $x=10$ را بنویسید.
۲. بلافاصله از *Replica 2* مقدار x را بخوانید.
۳. نتیجه را ثبت کنید.
۴. پس از چند ثانیه دوباره از *Replica 2* بخوانید.
۵. توضیح دهید چرا مقدار تغییر کرده است.

۲-۷ سناریوی ۲: خرابی یک replica

۱. هر سه replica را اجرا کنید.
۲. یکی از replica ها را متوقف کنید.
۳. یک عملیات *PUT* انجام دهید.
۴. رفتار سیستم را در مدل *eventual consistency* و مدل *strong consistency* مقایسه کنید.

۳-۷ سناریوی ۳: تعارض همزمان

۱. تقریباً همزمان روی دو replica مختلف برای یک کلید مقدار متفاوت بنویسید.
۲. اجازه دهید replica ها پیام های replication را رد و بدل کنند.
۳. مقدار نهایی هر replica را بررسی کنید.
۴. سیاست حل تعارض خود را توضیح دهید.

۴-۷ سناریوی ۴: تأثیر تأخیر شبکه

در سیستم خود یک تأخیر مصنوعی برای ارسال replication اضافه کنید.

- یک بار با تأخیر $0ms$
- یک بار با تأخیر $500ms$
- یک بار با تأخیر $2000ms$

سپس بررسی کنید:

- چه مدت طول می کشد تا همه replica ها همگرا شوند؟
- آیا کلاينت در این فاصله مقدار قدیمی می بیند؟
- تفاوت مدل *strong* و *eventual* در این حالت چیست؟

۸ متریک های مورد نیاز

در گزارش باید حداقل متریک های زیر آورده شوند:

- زمان پاسخ *PUT*

- زمان پاسخ *GET*
- زمان همگرایی replica ها
- تعداد replica های به روز شده پس از هر *PUT*
- تعداد خواندن های stale در آزمایش ها

۹ فناوری های مجاز

پیاده سازی می تواند با هر یک از روش های زیر انجام شود:

• *HTTP* ساده

• *JSON-RPC*

• *gRPC*

• *TCP* ساده

زبان پیشنهادی برای پیاده سازی *Go* است، اما در صورت نیاز می توانید همان سیاست زبانی تمرین های قبلی را اعمال کنید.

۱۰ ساختار پیشنهادی تحویل

HW3/

report.pdf

replica/

main.go

README.md

client/

```
main.go
README.md
configs/
  replica1.json
  replica2.json
  replica3.json
results/
  scenario1.txt
  scenario2.txt
  scenario3.txt
  scenario4.txt
```

۱۱ گزارش نهایی

گزارش باید شامل موارد زیر باشد:

- توضیح معماری سیستم
- نحوه اجرای replica ها
- نحوه اجرای client
- توضیح مدل consistency پیاده سازی شده
- توضیح سیاست versioning
- توضیح سیاست resolution conflict
- نتایج هر چهار سناریو
- جدول متریک ها

- تحلیل تفاوت *strong consistency* و *eventual consistency*

- مشکلات و محدودیت های پیاده سازی

۱۲ نمونه جدول گزارش

<i>Stale Reads</i>	<i>Convergence Time</i>	<i>PUT Latency</i>	<i>Model</i>	<i>Scenario</i>
...	...	...	<i>Eventual</i>	1
...	...	...	<i>Strong</i>	2

۱۳ معیار ارزیابی

۱-۱۳ بخش اول: گزارش مفهومی، 20 نمره

- توضیح درست replication و دلیل استفاده از آن: 4
- توضیح consistency و مشکل ناسازگاری replica ها: 4
- توضیح *strong consistency* و *eventual consistency*: 5
- توضیح های client-centric consistency مانند *read-your-writes*: 4
- تحلیل trade-off ها: 3

۲-۱۳ بخش دوم: پیاده سازی **store replicated** 40 نمره

- اجرای حداقل سه replica مستقل: 8
- پیاده سازی صحیح *GET* و *PUT*: 6
- replication بین replica ها: 8
- پیاده سازی مدل *eventual consistency*: 8

- پیاده سازی مدل *strong consistency* ساده شده: 7

- کیفیت کد و مدیریت خطا: 3

۳-۱۳ بخش سوم: *versioning* و *resolution conflict* 15 نمره

- نگهداری نسخه برای هر کلید: 5

- جلوگیری از *overwrite* شدن مقدار جدید با مقدار قدیمی: 4

- مدیریت *conflict* همزمان: 4

- توضیح سیاست انتخاب شده: 2

۴-۱۳ بخش چهارم: آزمایش و تحلیل، 25 نمره

- اجرای سناریوی ناسازگاری موقت: 5

- اجرای سناریوی خرابی *replica*: 5

- اجرای سناریوی تعارض همزمان: 5

- اجرای سناریوی تاخیر شبکه: 5

- تحلیل نتایج و جدول متریک ها: 5

۱۴ کسری نمره

موارد زیر باعث کسر نمره خواهند شد:

- اجرای *replica* ها فقط به صورت *object* یا *goroutine* داخل یک برنامه واحد

- نبودن ارتباط شبکه ای واقعی بین *replica* ها

- نبودن replication واقعی
- نداشتن سناریوی آزمایش
- نداشتن تحلیل و فقط ارائه خروجی خام
- نبودن README یا دستورات اجرای دقیق
- نداشتن versioning
- نامشخص بودن سیاست resolution conflict

۱۵ دستاوردهای تمرین

در پایان این تمرین باید بتوانید نشان دهید که:

- replication چگونه باعث افزایش دسترس پذیری و تحمل خطا می شود
- چرا replication مشکل consistency ایجاد می کند
- تفاوت consistency قوی و consistency eventual را در عمل مشاهده کرده اید
- می توانید یک store replicated ساده طراحی و پیاده سازی کنید
- می توانید conflict، read، stale و failure convergence را تحلیل کنید